GRIDDY

Project documentation

www.jlsola.com/GRIDDY · github.com/justinsola/GRIDDY · November 2026

Contents on the next page ↓

Contents

1. **Overview**
   What GRIDDY is and how it fits together

2. **Using GRIDDY**
   A visual walkthrough of the app

3. **The codebase**
   Pages, data model, state, key files

4. **Building locally**
   Setup, commands, before you push

5. **Deploying to GitHub Pages**
   Merge to deploy, the feature workflow, rollbacks

6. **Keeping it healthy**
   404.html, checks, license, and recent cleanup

↑ Contents

1. Overview

GRIDDY is a browser-only tool for building grid survey questions and exporting them to Qualtrics. There's no backend or database.

- Researchers design a grid in the editor and preview it live.
- Surveys are saved in the browser's `localStorage`. They can be backed up as JSON.
- Export produces a `.qsf` file that Qualtrics imports directly, or a JS snippet to paste.
- Respondents answer inside Qualtrics. Answers are stored as JSON in embedded-data fields (`__js_GridAssignments`, or `__js_GridPrefills` / `__js_GridResponses` in experimental mode), in Qualtrics' new survey-taking experience and in the older layouts.

```
Editor (ConfigPanel) → EditorContext (state) → PreviewPanel
                                ↓
          surveysApi → localStorage    qualtricsExport → .qsf / JS → Qualtrics
```

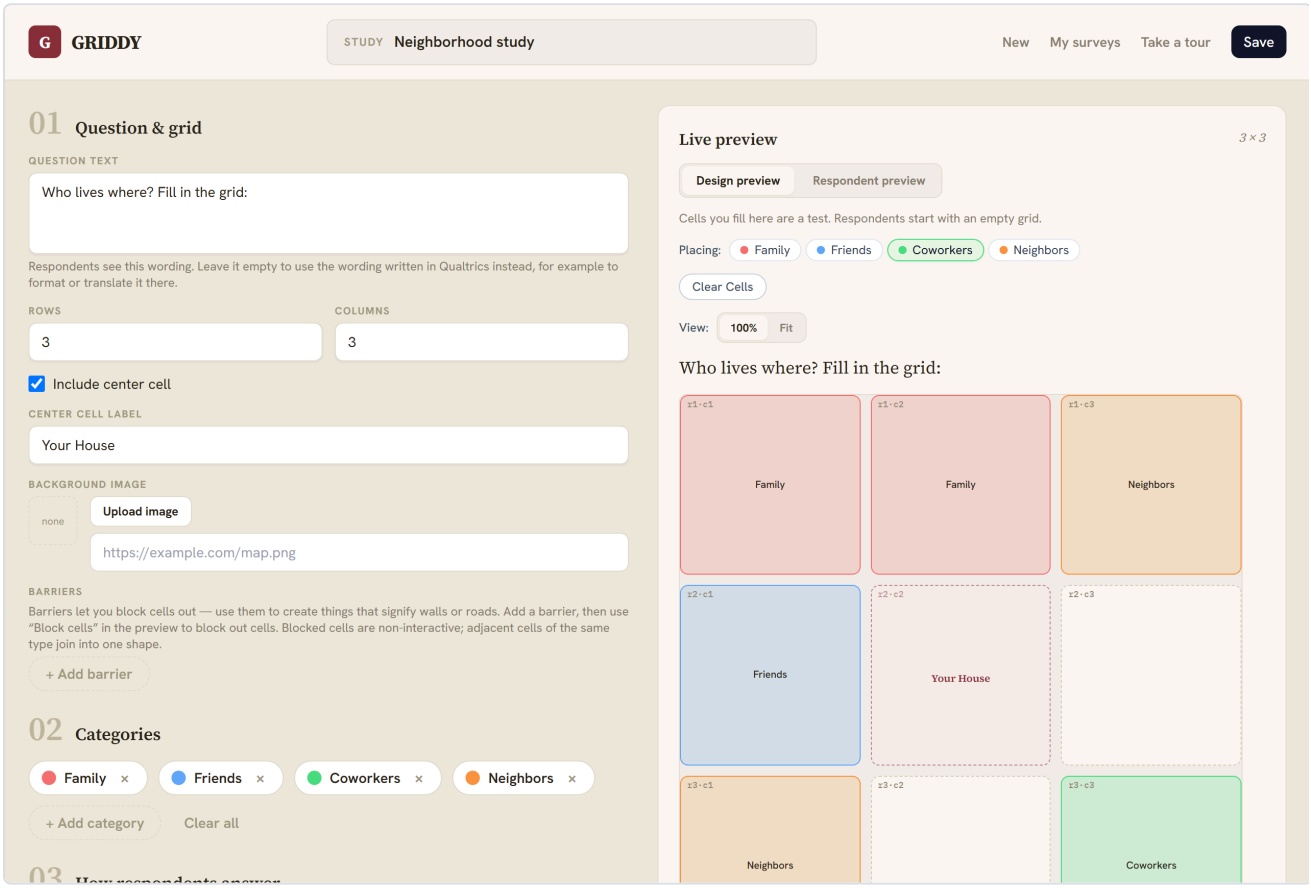
Stack: React 18, TypeScript, Vite, Tailwind, React Router, Vitest. Hosted on GitHub Pages as static files. Licensed under the MIT License.

↑ Contents

2. Using GRIDDY

The editor

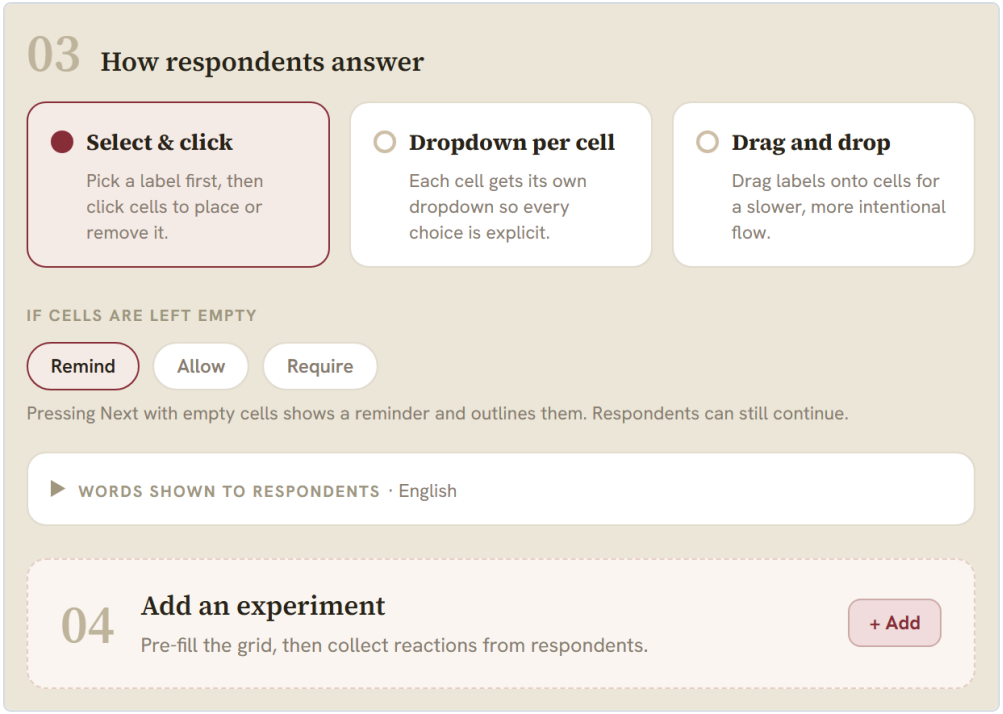
Settings on the left, live preview on the right. Pick a category in the Placing row, then click cells to try the grid. These test fills are not saved: respondents start with an empty grid. A line at the top of the design preview says what a click does; in an experiment with fixed or shuffled pre-fills, clicks change the pre-fills every respondent sees. Both previews show the grid at full size, with Fit as an option. Name the survey at the top and click Save. If the survey was renamed, or changed since it was opened or last saved (in another tab or window, or by importing a backup), Save asks whether to replace the saved copy or save it as a new survey.



The editor at `/`: question & grid settings (01), categories (02), and the live preview at full size, with test fills.

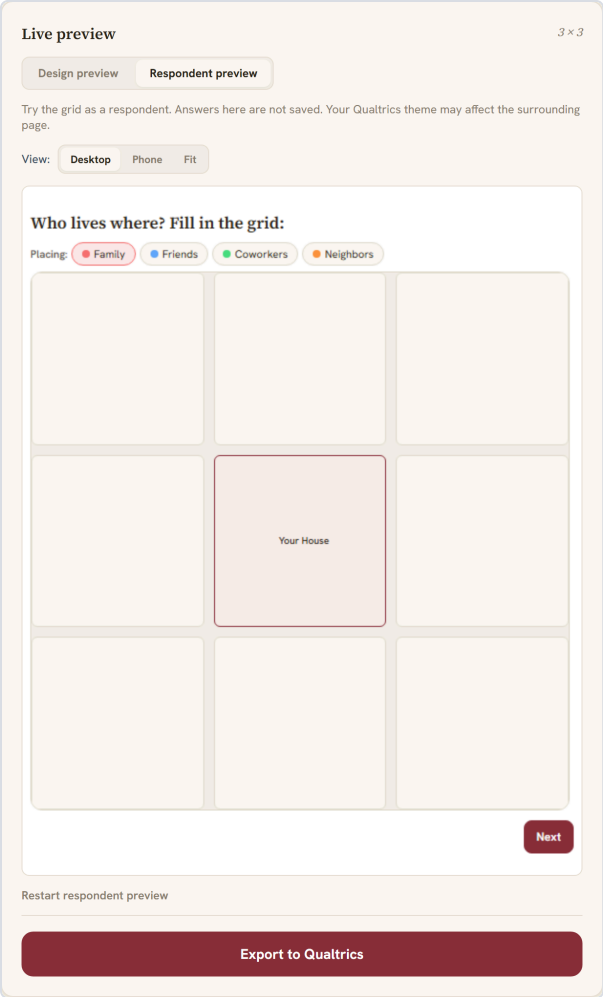
How respondents answer, and experiments

Section 03 picks the input method. Under **Words shown to respondents** it can also reword the texts GRIDDY adds around the grid, such as "Placing:", the empty-cell reminder and Next, which are English by default. Section 04 turns on experimental mode, where the grid is pre-filled and respondents react to it.



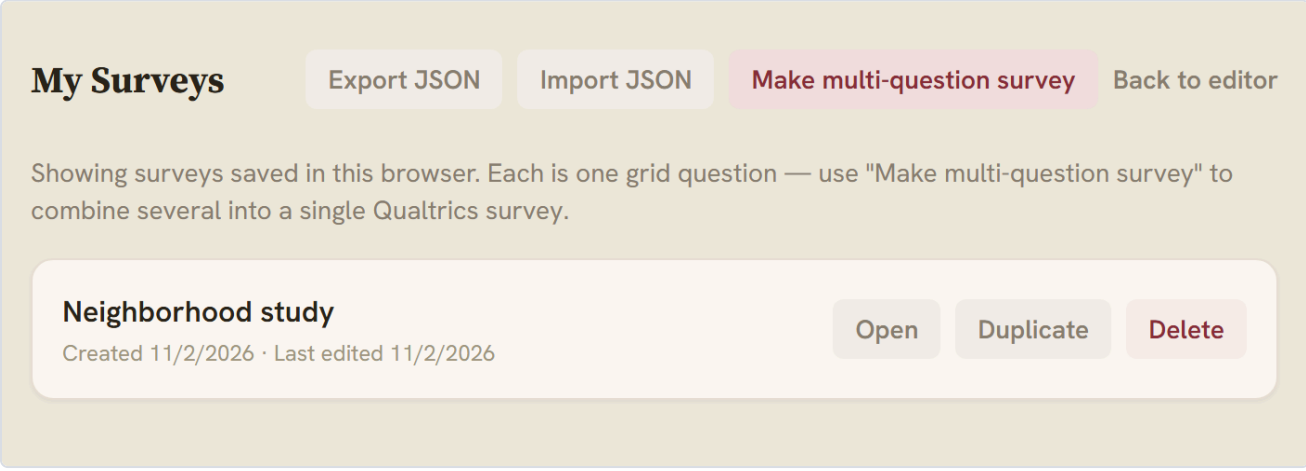
Respondent preview

Runs the real exported script, with the grid at full size. **View** switches between Desktop (the default), Phone (a 375 px screen, with the scroll hint and edge arrows) and Fit (zoomed out). Answers here aren't saved, and your Qualtrics theme may change the page around the grid.



The Respondent preview tab in its Desktop view, at full size. Phone and Fit are the other views.

My Surveys

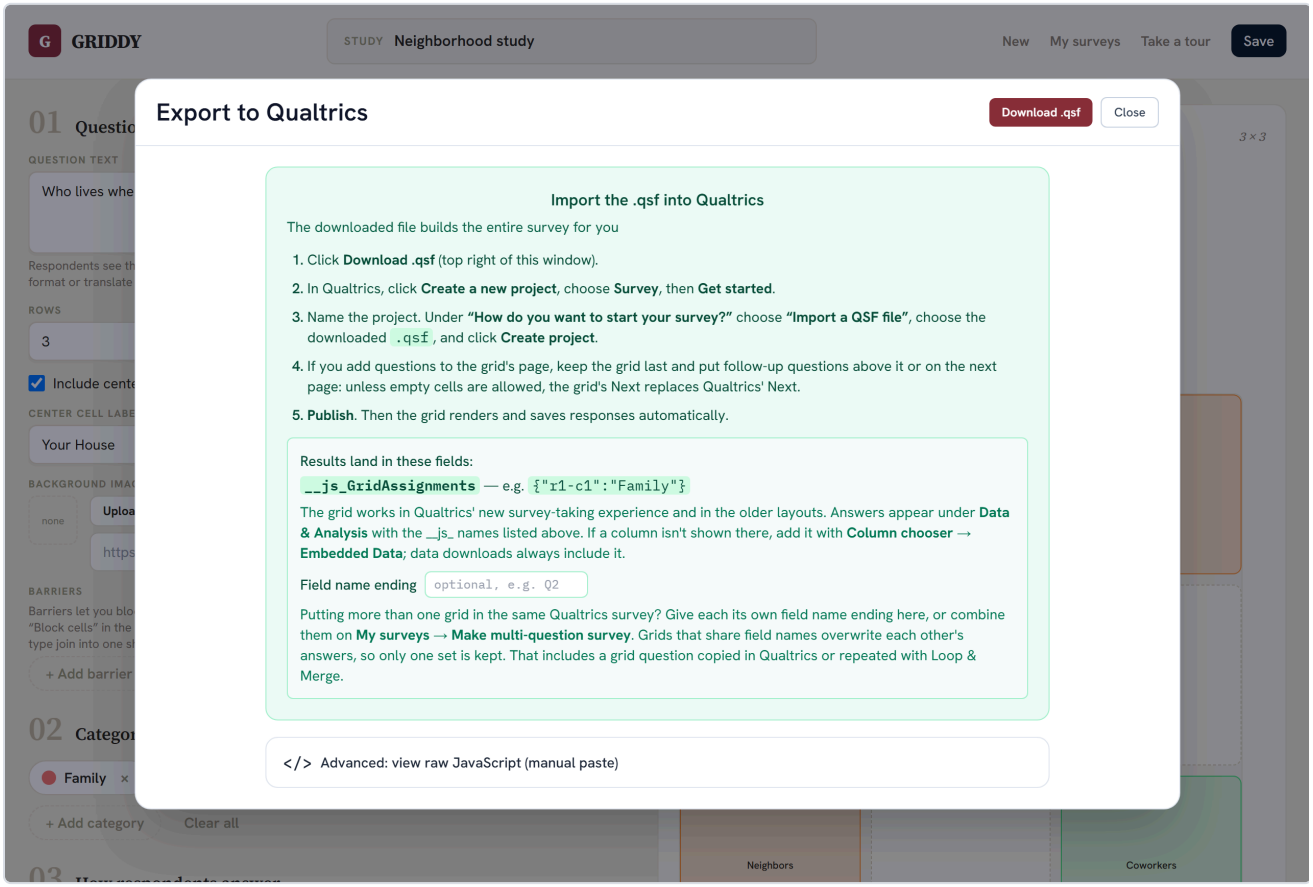


`/history` : surveys saved in this browser, with Open, Duplicate and Delete, and JSON backup and restore.

Export to Qualtrics

Click **Export to Qualtrics**, then **Download .qsf**, and import that file in Qualtrics. The dialog lists the steps and the fields the answers are saved to, as Qualtrics records them: `__js_GridAssignments`, or `__js_GridPrefills` and `__js_GridResponses`. Each grid gets a page of its own; if you add questions, keep the grid as the last question on its page and put follow-up questions above it or on the next page.

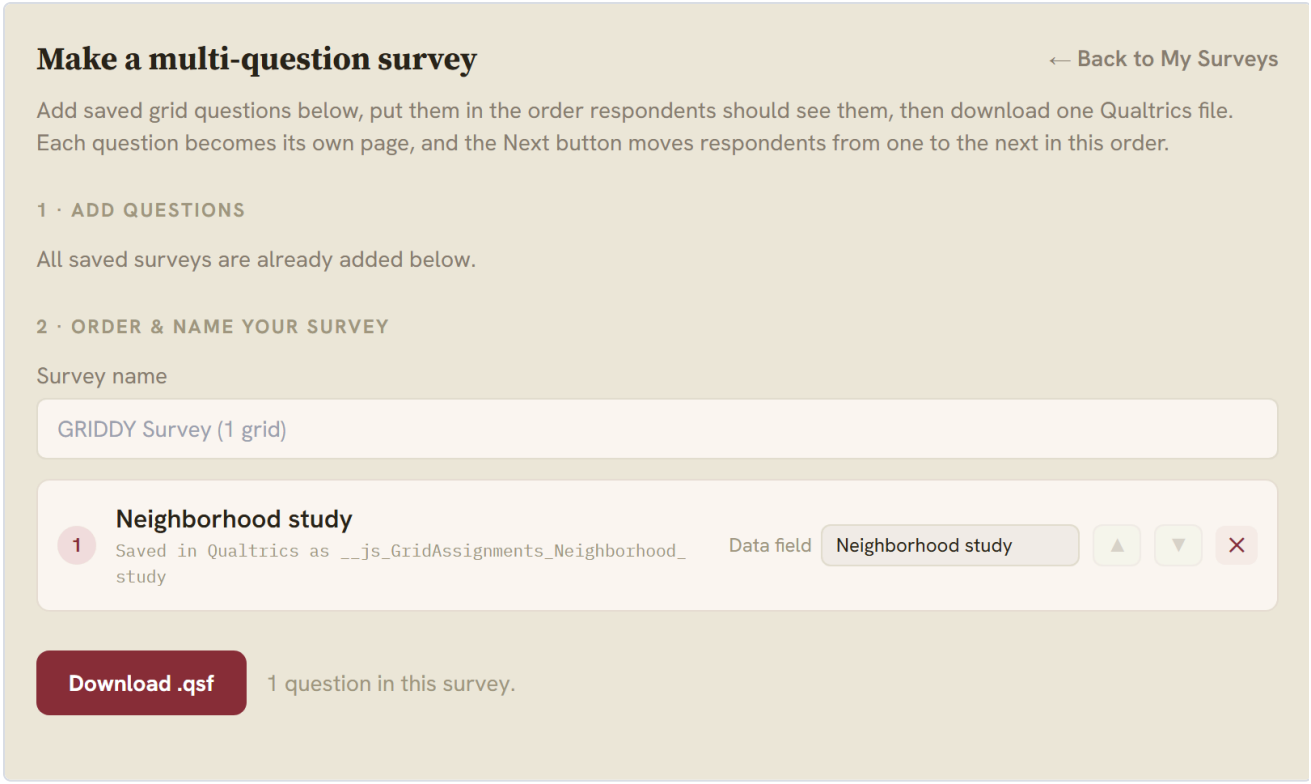
Exports from older versions of GRIDDY save to fields without `__js_`. They record nothing in Qualtrics' new survey-taking experience, the default for surveys created since June 2026: export and import them again.



The export dialog, listing the recorded name `__js_GridAssignments`. A raw-JavaScript option is tucked away under "Advanced".

Combining several grids into one survey

From My Surveys, click **Make multi-question survey**. Add saved grids, order them, name each data field, then download one `.qsf`.



`/history/combine` : each grid becomes its own page in Qualtrics, and each row shows the field it is saved in.

↑ Contents

3. The codebase

Pages

URL	FILE	PURPOSE
`/`	`App.tsx`	The editor and live preview
`/history`	`pages/HistoryPage.tsx`	My Surveys: open, delete, JSON backup
`/history/combine`	`pages/CombineSurveysPage.tsx`	Combine several surveys into one `.qsf`

Data model (`src/grid-types.ts`)

A survey is a `GridConfig` with four parts: **layout** (rows, cols, center cell, barriers), **tuning** (sizes), **survey** (categories, selection mode, and any own wording for the respondent texts) and optional **experimental** (pre-filled grid + response labels). Cells are keyed `"r{row}-c{col}"`, 1-based.

State (`src/EditorContext.tsx`)

One reducer holds the survey being edited. Components read it with `useEditor()` and change it by dispatching actions such as `updateLayout`, `updateSurvey`, `setConfig` or `newSurvey`.

Key files

`components/ConfigPanel.tsx`	Left-hand editor UI
`components/PreviewPanel.tsx`	Live preview and Export modal
`components/RespondentPreview.tsx`	Runs the real exported script
`lib/qualtricsExport.ts`	Builds the Qualtrics JS and `.qsf`
`lib/surveysApi.ts`	Save, load, duplicate and import surveys (localStorage)
`lib/respondentTexts.ts`	The texts GRIDDY shows respondents, in English, and their placeholders
`lib/imageInput.ts`	Compresses uploaded images into the survey

Adding a new setting

Add it to the type → give it a default in `createDefaultConfig` and `normalizeConfig` → add a control in `ConfigPanel` → render it in `PreviewPanel` and in `qualtricsExport.ts` → add a test.

> The Qualtrics script is plain JavaScript, not React, so visual features have to be built in both places. Use the respondent preview to check that they match.

↑ Contents

4. Building locally

Requires Node 22+ and git. No environment variables are needed.

```
git clone https://github.com/justinsola/GRIDDY.git
cd GRIDDY
npm install
npm run dev        # http://localhost:5173
```

COMMAND	DOES
`npm run dev`	Dev server with hot reload
`npm run build`	Type-check, then bundle to `dist/`. This is what the Pages deploy runs.
`npm run preview`	Serve the built `dist/` locally
`npm test`	Run the Vitest tests
`npm run lint`	ESLint

Before pushing: run `npm run build` (and ideally `npm test` and `npm run lint`). If it fails locally, the deploy fails too. Unused variables count as build errors.

To see exactly what will be deployed, build with the live site's path and preview it at `http://localhost:4173/GRIDDY/`:

```
BASE_PATH=/GRIDDY/ npm run build
npm run preview
```

Saved surveys are stored per site: localhost and the live site keep separate lists. Use Export/Import JSON to move them.

↑ Contents

5. Deploying to GitHub Pages

A GitHub Actions workflow (`.github/workflows/pages.yml`) publishes the site. **Merging to `main` is the deploy.** There's no separate deploy command.

PUSH TO…	RESULT
`main`	The workflow builds the app and the live site updates in a minute or two
Any other branch or PR	Only the CI checks run. The live site is untouched.

The site is served at **https://www.jlsola.com/GRIDDY/**: a project site on GitHub Pages lives at the owner's Pages domain plus the repository name. The workflow passes the repository name to the build as `BASE_PATH` , so renaming the repository moves the site with no code change. Under the repo's **Settings** → **Pages**, the source must stay **GitHub Actions**.

Adding a feature

```
git switch main && git pull
git switch -c feature/my-change
# ...build it, check npm run build...
git add -A && git commit -m "My change"
git push -u origin feature/my-change
```

1. Open a pull request into `main` on GitHub.
2. Wait for the green check, **CI / Build, test, lint** (GitHub Actions). Red means something failed: click **Details**.
3. Merge the PR. The live site updates automatically; watch the run under **Actions** → **Deploy to GitHub Pages**.

If the live site breaks

- **Via git:** `git revert <commit>` on `main` and push. The workflow deploys a build without the change.
- **Redeploy the last good build:** under **Actions** → **Deploy to GitHub Pages**, open the last successful run and choose **Re-run all jobs**.

Everything needed to deploy lives in this repository and the owner's GitHub account. No third-party dashboard is involved.

↑ Contents

6. Keeping it healthy

404.html

The app has client-side routes, so there's no real `/history` file. GitHub Pages serves `404.html` for any path that isn't a real file, and the deploy workflow makes `404.html` a copy of `index.html`, so loading or refreshing any page works. Two pieces of code keep routes and assets on the `/GRIDDY/` path: `vite.config.ts` sets Vite's `base` from `BASE_PATH`, and `src/main.tsx` gives React Router the same path as its `basename`. Don't hard-code absolute URLs; use router links and relative asset imports.

Checks

`npm run build`, `npm test` and `npm run lint` all pass on `main`. GitHub Actions (`.github/workflows/ci.yml`) runs all three on every PR and every push to `main`. If one fails, something you changed caused it.

CI installs with `npm ci`, so when you change dependencies, commit the updated `package-lock.json` too.

License

GRIDDY is released under the MIT License, Copyright (c) 2026 Justin Lucas Sola. The JavaScript that GRIDDY exports into a Qualtrics survey carries the copyright and permission notice as a comment, so a survey that embeds it already satisfies the license's one condition.

Cleanup done (October 2026)

- Fixed the 404 when refreshing `/history` and `/history/combine`.
- Updated an outdated test and fixed all lint errors and warnings.
- Added GitHub Actions CI and synced `package-lock.json`.
- Removed the unused `LayoutTab.tsx` and `SurveyTab.tsx`, and the unused `localforage` and `react-webcam` packages.
- Transferred the repository to `justinsola/GRIDDY`, adopted the MIT License, added `CITATION.cff`, and gave the README its mascot.
- Moved hosting from Vercel to GitHub Pages at www.jlsola.com/GRIDDY.

Changes in November 2026

- The grid keeps its size on phones and scrolls sideways. Answers are recorded in Qualtrics' new survey-taking experience too, under `__js_` field names.
- Save asks before replacing a survey, My Surveys has Duplicate, previews show the grid at full size, and each grid can reword the texts respondents see.